



Mobile Design - Android

This content is protected and may not be shared, uploaded, or distributed.

Copyright © 2017-2025 Marco Papa and Sunny Patel

Outline

- Android – background, architecture and features
- Android Studio basics
- Basic concepts – Resources, Activity, Services, etc.
- Event handling
- HTTP networking and JSON parser
- Running application on an Android device/Emulator
- References

Background

- Android is an open source and Linux-based Operating System for smartphones.
- Android Inc. was founded in Palo Alto, California in October 2003 by Andy Rubin, Rich Miner, Nick Sears, and Chris White.
- Android was pitched as a handset operating system that would rival Symbian and Microsoft Windows Mobile.
- In July 2005, Google acquired Android Inc. for at least \$50 million. Its key employees, including Rubin, Miner and White, joined Google as part of the acquisition
- On November 5, 2007, the *Open Handset Alliance*, a consortium of technology companies including Google, device manufacturers such as HTC, Motorola and Samsung, wireless carriers such as Sprint and T-Mobile, and chipset makers such as Qualcomm and Texas Instruments, unveiled itself, with a goal to develop "the first truly open and comprehensive platform for mobile devices".
- Developers need to develop only once for Android and the applications run across all Android devices.
- The first beta version of the Android Software Development Kit (SDK) was released by Google in 2007 whereas the first commercial version, Android 1.0, was released in September 2008.
- The first commercially available smartphone running Android was the HTC Dream, also known as T-Mobile G1, announced on September 23, 2008.

Features

- Open Source
- Larger Developer Community
- High Marketing
- Rich Development Environment
- Inter App Integration Feature
- High Security
- Reduced Cost of Development
- Higher Success Ratio
- See <https://www.android.com/>

Categories of Applications



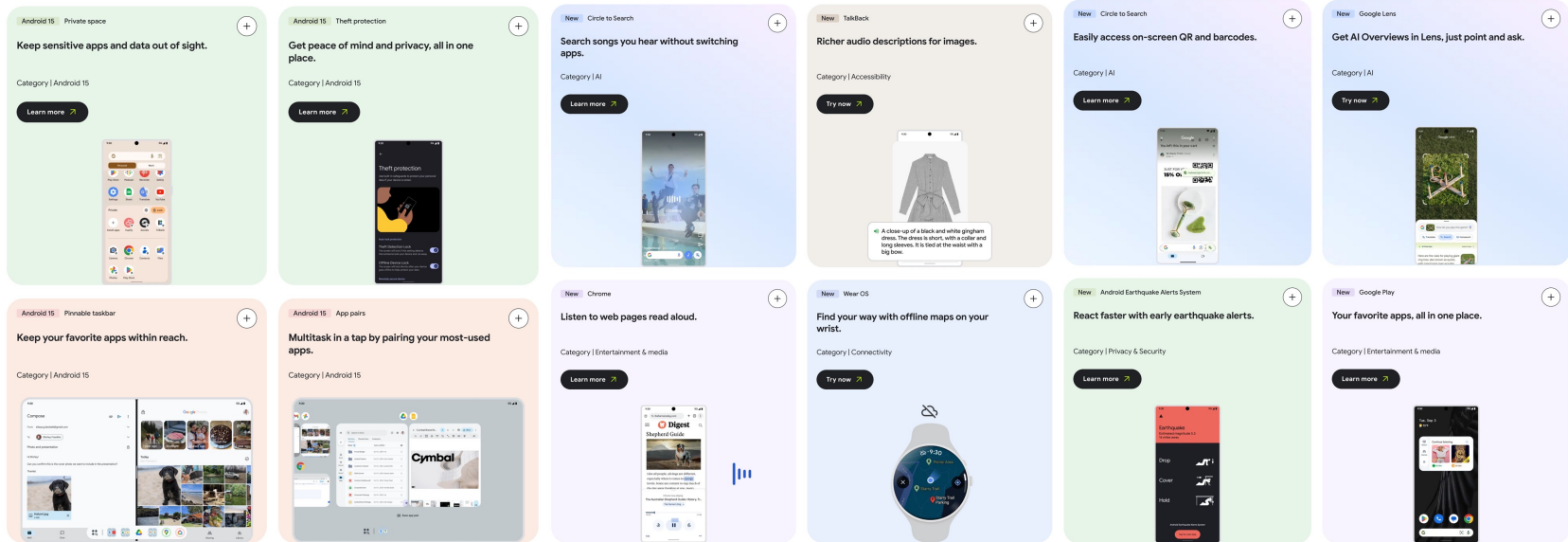
History

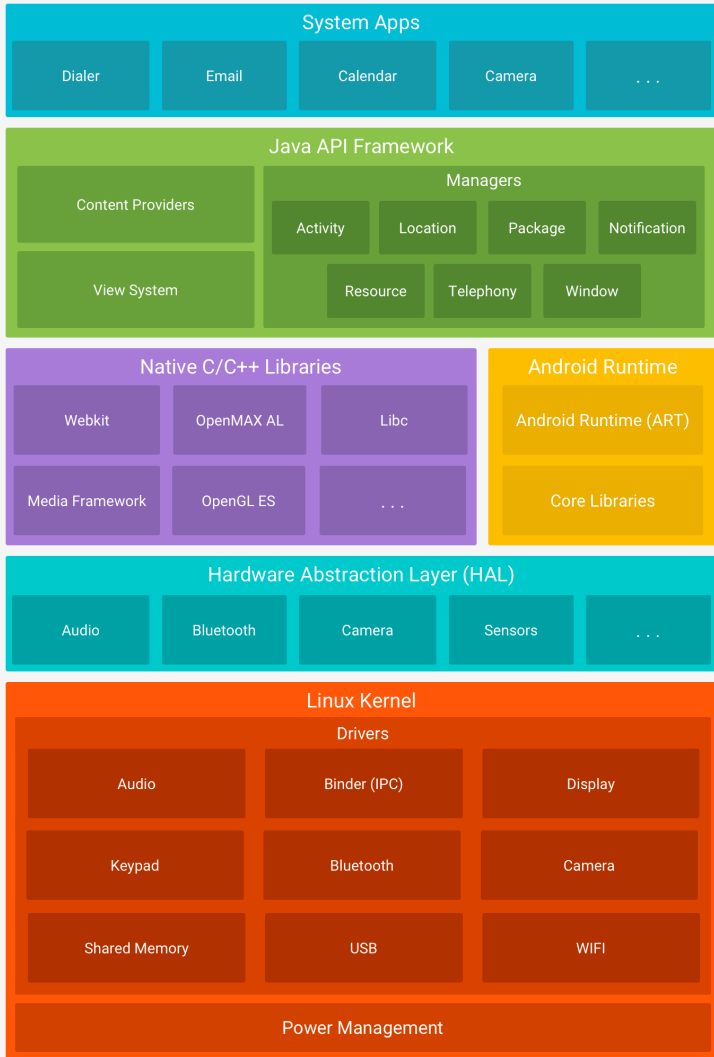
Android Version	Code name	API level
1.0	APPLE PIE	1
1.1	BANANA BREAD	2
1.5	CUPCAKE	3
1.6	DONUT	4
2.0, 2.1	ECLAIR	5, 6, 7
2.2	FROYO	8
2.3	GINGERBREAD	9 and 10
3.0, 3.1, 3.2	HONEYCOMB	12 and 13
4.0	ICE CREAM SANDWICH	14, 15
4.1, 4.2, 4.3	JELLYBEAN	16, 17 and 18
4.4	KITKAT	19, 20
5.0, 5.1	LOLLIPOP	21, 22
6.0	MARSHMALLOW	23
7.0, 7.1	NOUGAT	24, 25
8.0, 8.1	OREO	26, 27
9.0	PIE	28
10.0	Android 10	29
11.0	Android 11	30
12.0	Android 12	31
13.0	Android 13	32
14.0-15.0	Android 14 Android 15	33-34

Android 15



- See: <https://www.android.com/new-features-on-android/>



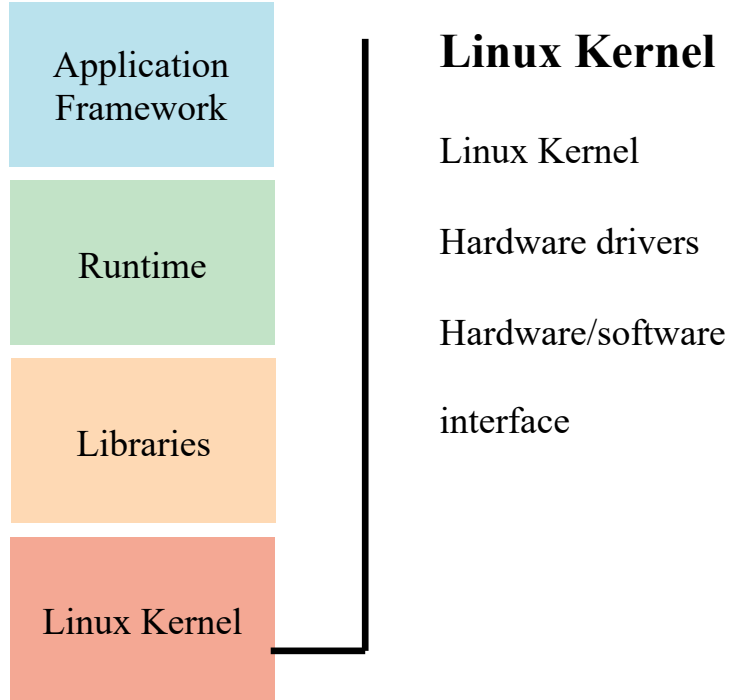


Architecture

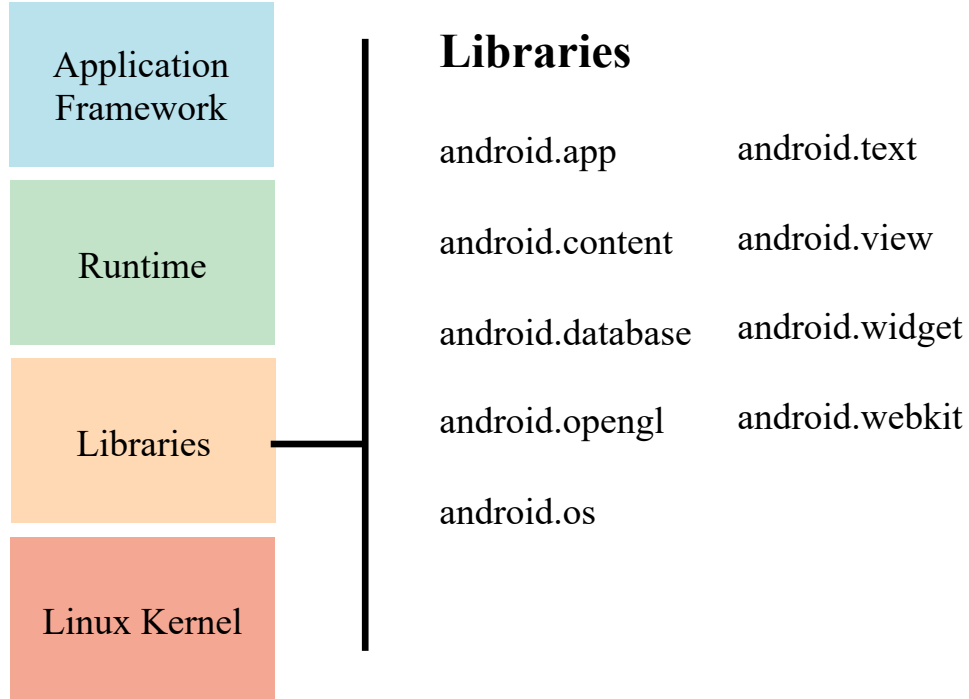
Android operating system is a stack of software components which is roughly divided into six sections and five main layers as shown in the architecture diagram.

<https://developer.android.com/guide/platform>

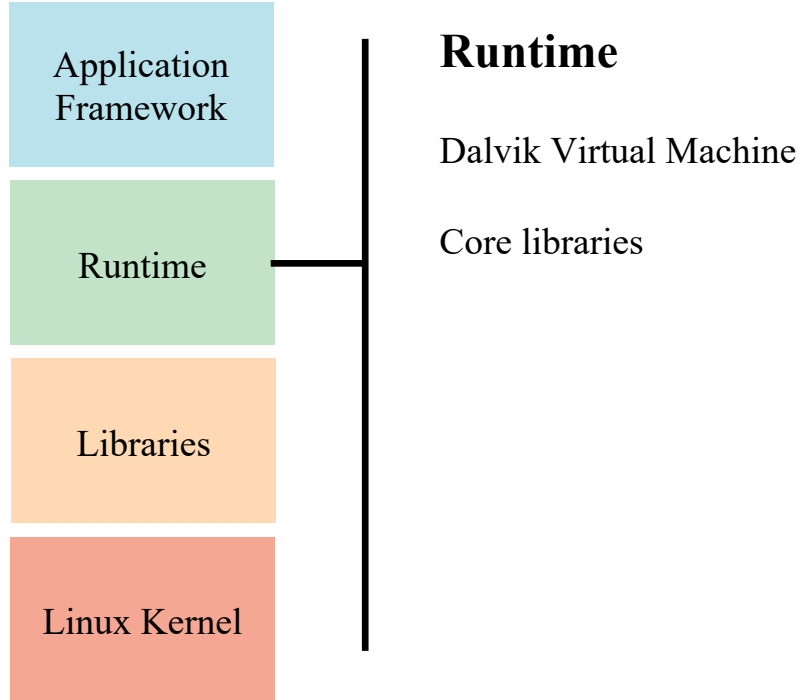
Architecture



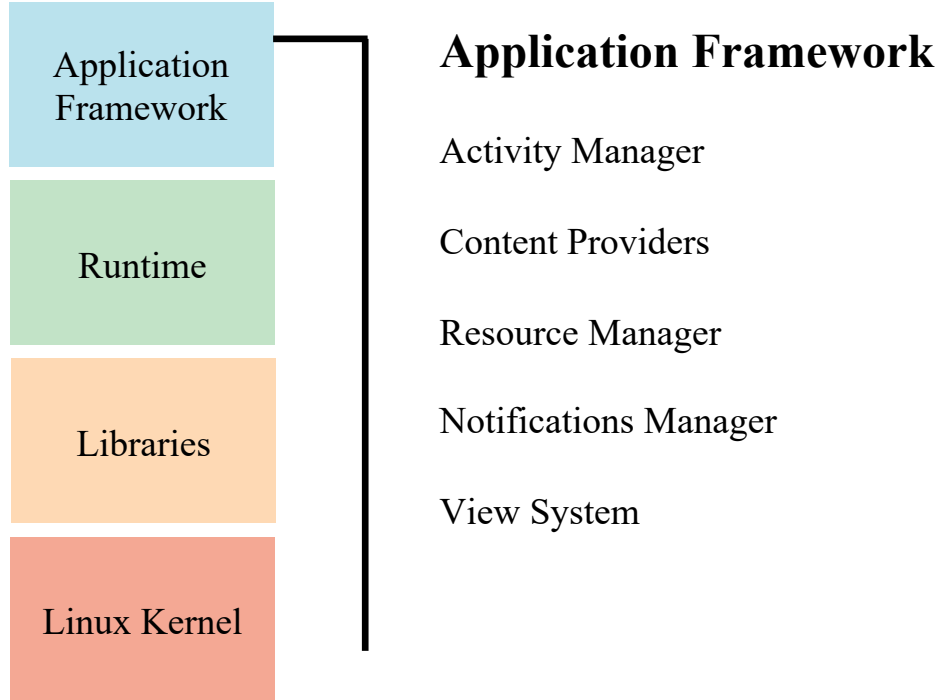
Architecture



Architecture



Architecture



Android Studio Basics

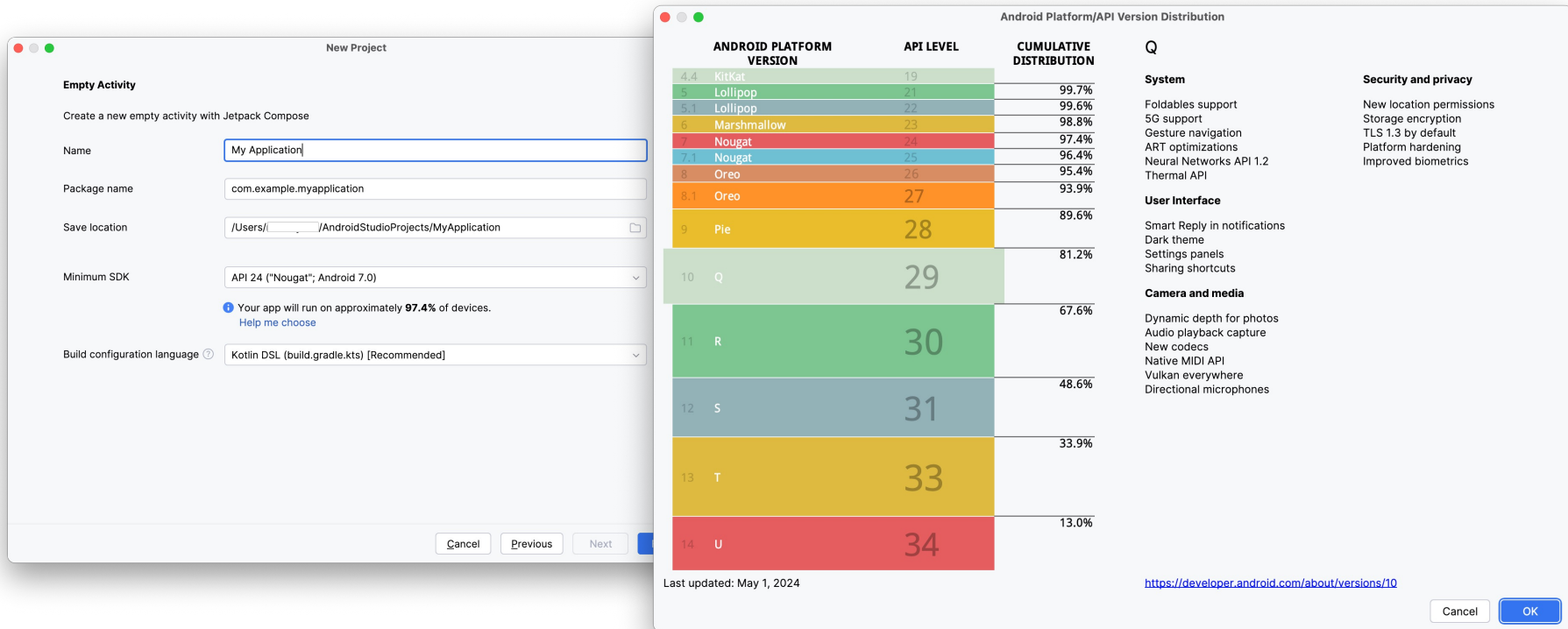
- Setup
- Create a new project
- Get familiar with Android Studio
- Design UI in XML using drag and drop in xml resource layout editor
- Control UI using Activity
- Change Activity properties in Manifest file
- Run your app in the emulator
- Download at <https://developer.android.com/studio/>

New Version Numbering

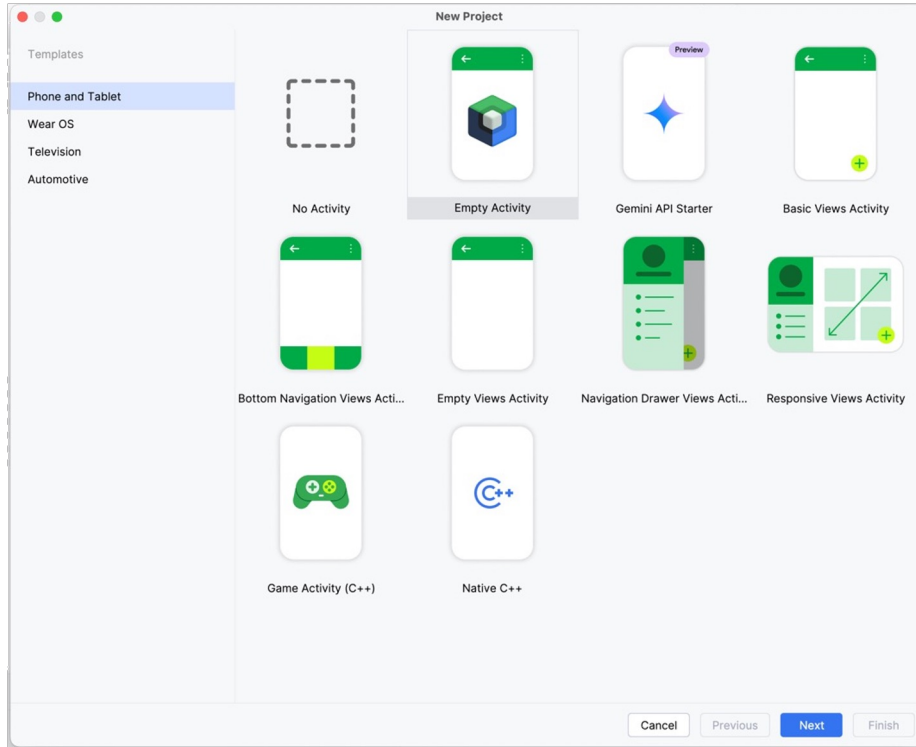
- Changed the version numbering system for Android Studio to more closely align with [IntelliJ IDEA](#), the IDE that Android Studio is based on.
- In the previous numbering system, latest release would have been numbered as *Android Studio 4.3* or version *4.3.0.1*. With the new numbering system, it is now *Android Studio - Arctic Fox | 2020.3.1*, or version *2020.3.1*.
- Latest version **Android Studio Meerkat | 2024.3.1 Patch 1**

IntelliJ Version	Old Name	Old - Number System	New - Year System	New Version Name
2020.3	4.3	4.3.0	2020.3.1	Arctic Fox 2020.3.1

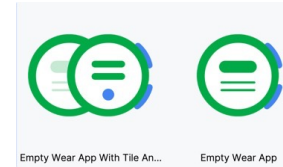
Android Studio - Create a new project



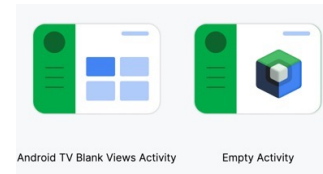
Android Studio - Create a new project (cont'd)



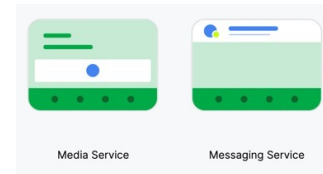
WearOS



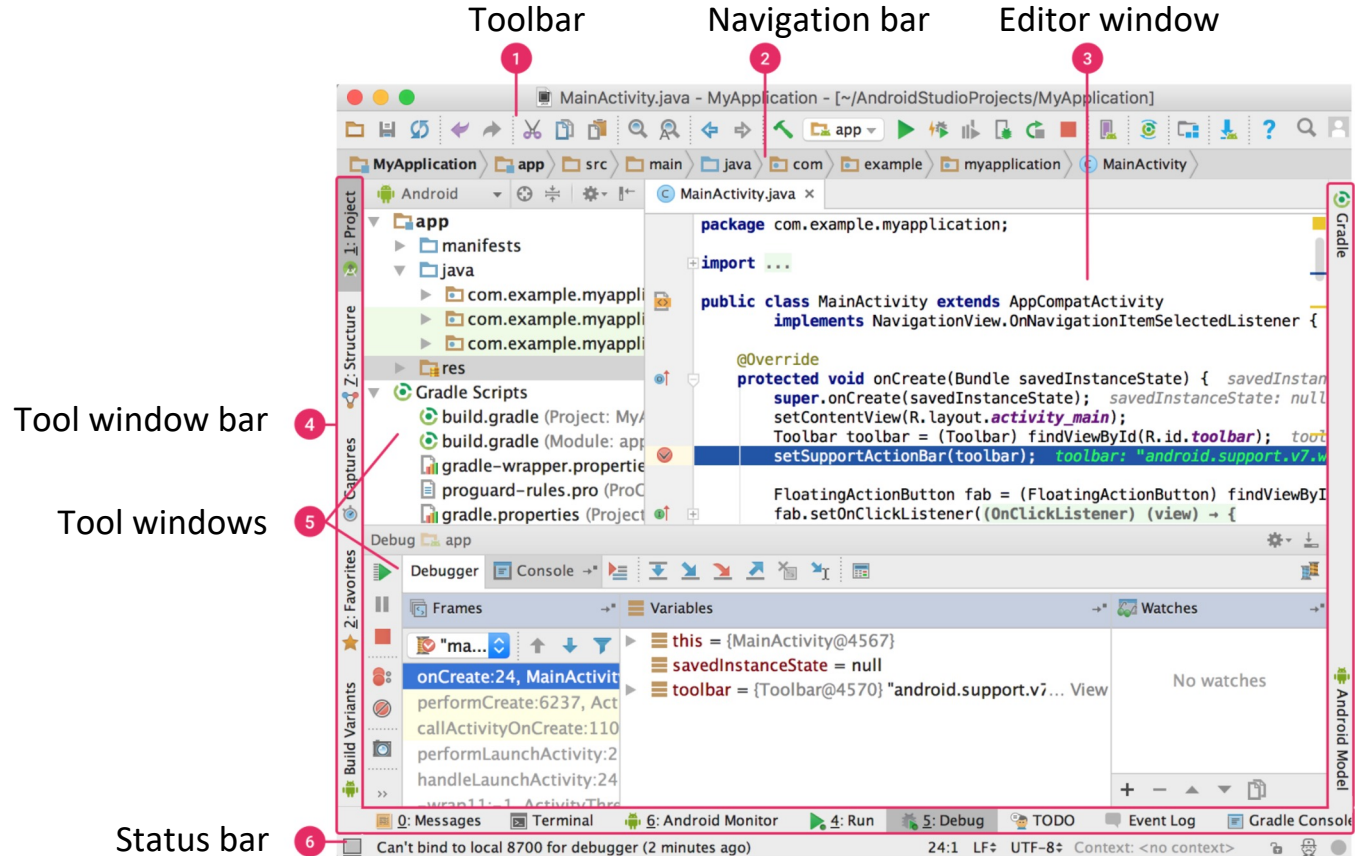
Android TV



Android Auto



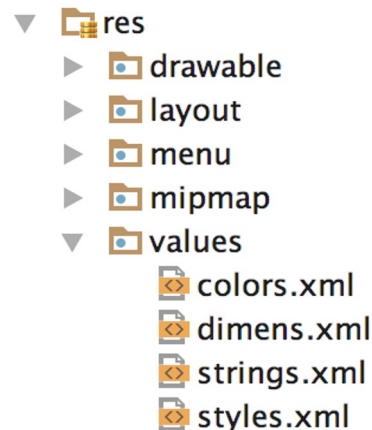
Get familiar with Android Studio



Resources

- Types of resources: anim, color, drawable, layout, menu, raw, values, xml.
- Alternately, you can create a new directory in res/ named in the form **<resources_name>-<config_qualifier>**
- To access *res/drawable/myimage.png* and set an ImageView you will use following code –

```
ImageView imageView = (ImageView)findViewById(R.id.myimageview);  
imageView.setImageResource(R.drawable.myimage);
```



Activity

- An **activity** represents a single screen with a user interface.
- Android system initiates its program within an Activity starting with a call to *onCreate()* callback method.

Life-cycle of an Activity:

1. **onCreate():** This is the first callback and called when the activity is first created.
2. **onStart():** This callback is called when the activity becomes visible to the user.
3. **onResume():** This is called when the user starts interacting with the app.

Activity (cont'd)

4. **onPause():** This callback is called when the current activity is being paused and the previous activity is being resumed. During this time, it does not receive user input or execute any code.
5. **onStop():** This callback is called when the activity is no longer visible.
6. **onDestroy():** This callback is called before the activity is destroyed by the system.
7. **onRestart():** This callback is called when the activity restarts after getting stopped.

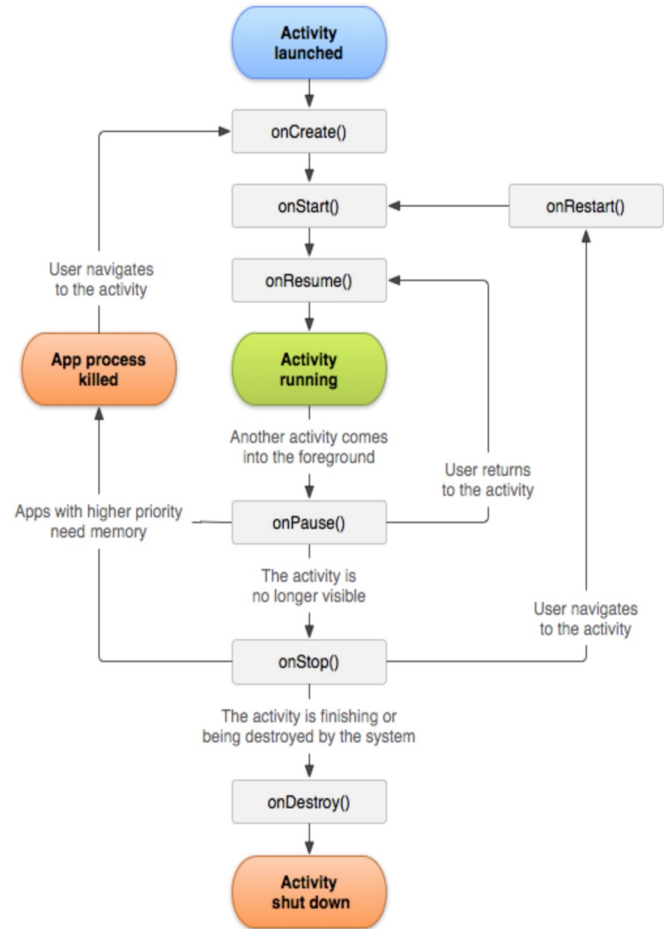


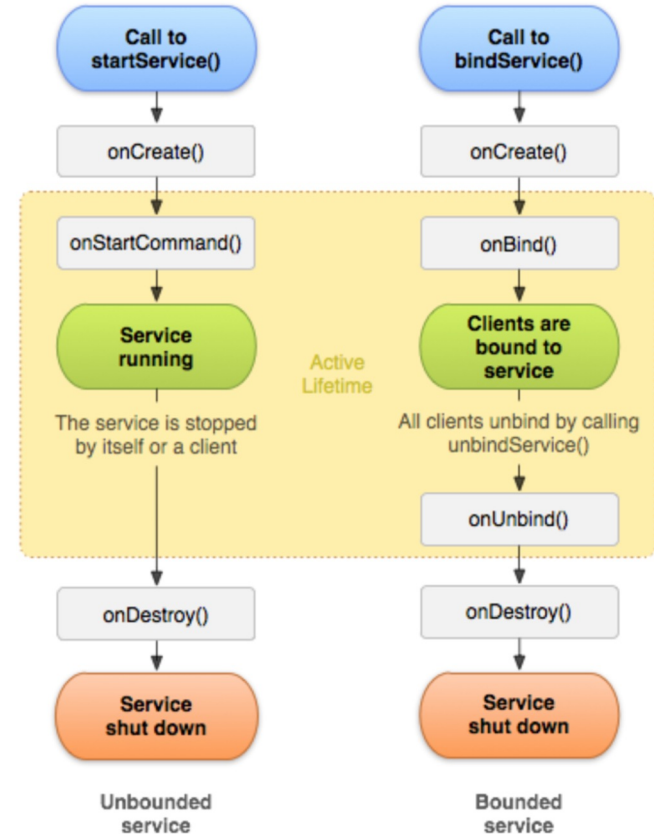
Figure 1. A simplified illustration of the activity lifecycle.

Services

A **service** runs in the background to perform a task without user interaction and keeps working even if application is destroyed.

A service can essentially take two states –

1. **Started:** A service is started when an activity starts it by calling *startService()*.
2. **Bound:** A service is bound when an application component binds to it by calling *bindService()*.



Broadcast Receivers

- A Broadcast Receiver reacts to the system generated messages or the messages generated from other applications. These messages are called **events** or **intents**.
- The following steps are needed to use and respond to events using Broadcast Receiver:
 1. **Creating the Broadcast Receiver:** A receiver can be created in an Activity by extending the abstract class BroadcastReceiver.

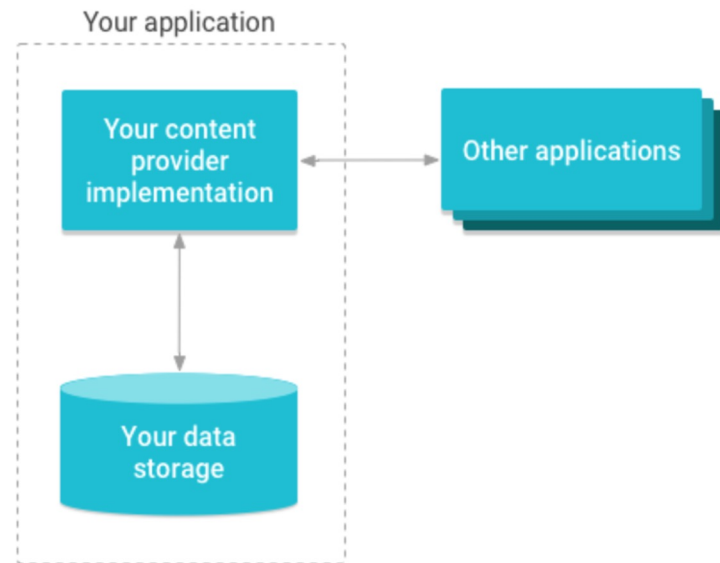
```
public class MyReceiver extends BroadcastReceiver {  
    ...@Override  
    ... public void onReceive(Context context, Intent intent) {  
        ... Toast.makeText(context, "Intent Detected (Power Connected)", Toast.LENGTH_LONG).show();  
        ... }  
    ... }
```

1. **Registering Broadcast Receiver:** A receiver can be registered in Android Manifest file.

```
<receiver android:name="MyReceiver">  
    <intent-filter>  
        <action android:name="android.intent.action.ACTION_POWER_CONNECTED">  
        </action>  
    </intent-filter>  
</receiver>
```

Content Providers

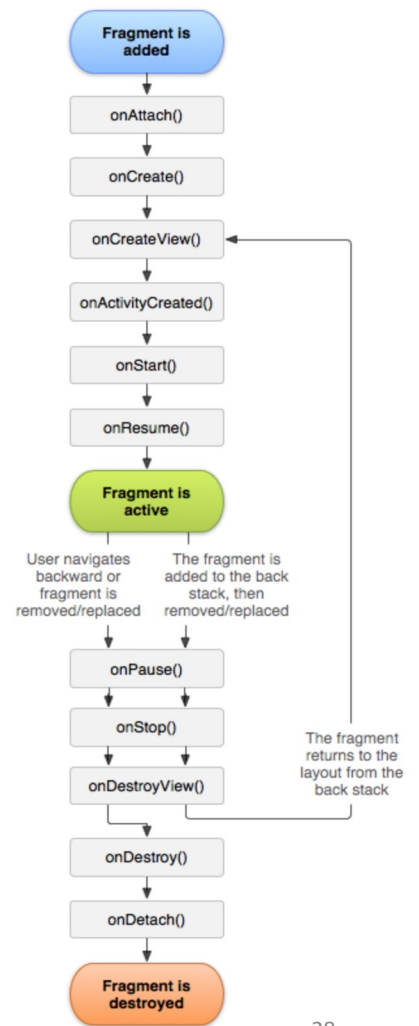
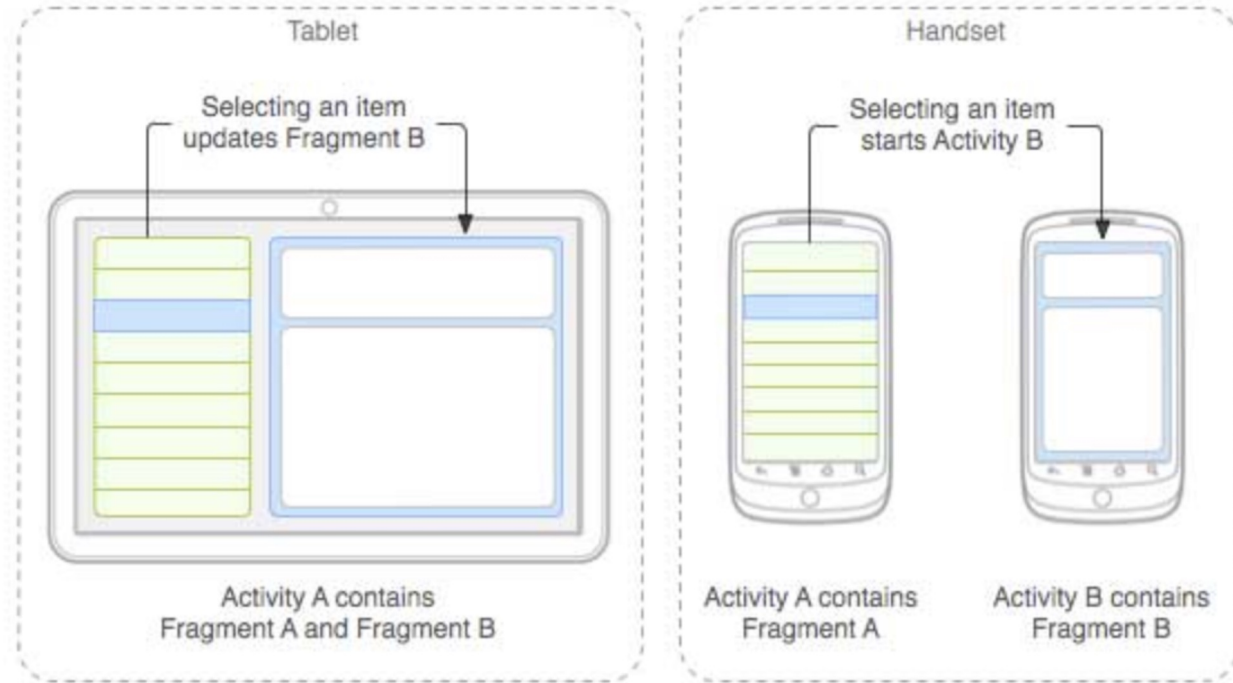
- Often data from one application is needed in another application. A content provider supplies data on demand.
- Using content providers, you can have centralized data and other applications can access it whenever needed. It behaves more like a database which can be queried, updated, inserted into like any other DB.



Fragments

- A Fragment is often used for a more modular design- it may or may not be a part of an Activity. It can also be considered as a **sub-activity**.
- A fragment can run without a user interface as well.
- A fragment has its own layout and its own behavior with its own life cycle callbacks.
- It is possible to dynamically add fragments to a running activity.
- A fragment once created, can be used in multiple activities.
- A fragment's life cycle is closely knitted with its parent activity's life cycle. When the activity is paused or stopped, fragments also replicate the same status.
- Usually, tablets have multiple fragments running side by side, in one activity (example on next slide)

Life cycle of a Fragment



Intents & Intent Filters

- An *Android Intent* is a messaging object you can use to request an action from another app component. The 3 main use cases are:
 - Starting an Activity
 - Starting a Service
 - Delivering a broadcast

The primary information contained in an Intent Object is the following:

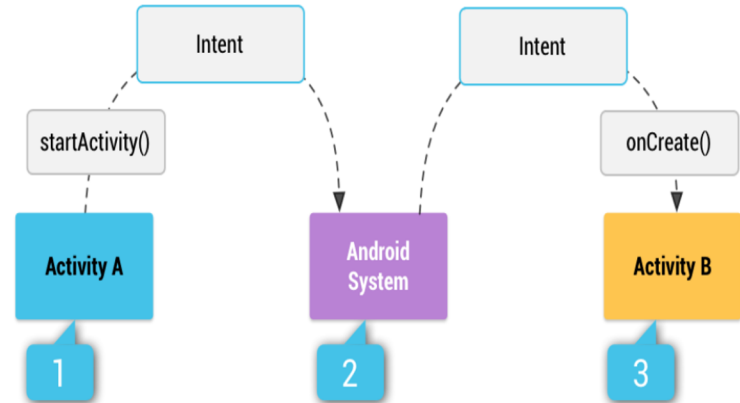
- **Component name:** The name of the component to start.
- **Action:** A string that specifies the generic action to perform (such as *view* or *pick*). This is mandatory part of the Intent object.
- **Data:** The URI (a [Uri](#) object) that references the data to be acted on and/or the MIME type of that data.

Intents & Intent Filters (cont'd)

- **Category:** A string containing additional information about the kind of component that should handle the intent. The category is an optional part of Intent object.
- **Extras:** This will be in key-value pairs for additional information that should be delivered to the component handling the intent.
- **Flags:** Flags function as metadata for the intent. The flags may instruct the Android system how to launch an activity and how to handle the intent.

Example:

```
Intent i = new Intent(FirstActivity.this,  
SecondActivity.class);  
startActivity(i); // Starts SecondActivity
```



UI Layouts

A layout, usually defined in XML, describes the structure and placement of UI components for an Activity. You can declare a layout in two ways:

- **Declare UI elements in XML.** Android provides a straightforward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts.
- **Instantiate layout elements at runtime.** Your application can create View and ViewGroup objects (and manipulate their properties) programmatically.

Next slide: Example code for defining a TextView and a Button in Linear Layout.

UI Layouts (cont'd)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a TextView" />
    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a Button" />
</LinearLayout>
```

Loading the XML resource in Java:

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main_layout);
}
```

UI Layouts (cont'd)

Common Layouts

Linear Layout



A layout that organizes its children into a single horizontal or vertical row. It creates a scrollbar if the length of the window exceeds the length of the screen.

Relative Layout



Enables you to specify the location of child objects relative to each other (child A to the left of child B) or to the parent (aligned to the top of the parent).

Web View



Displays web pages.

Input Controls

Input Controls controls are the interactive components in your app's user interface. Android provides a wide variety of controls you can use in your UI, such as buttons, text fields, seek bars, check box, zoom buttons, toggle buttons, and many more.

Common controls include:

1. **TextView:** This control is used to display text to the user.
2. **EditText:** This is a predefined subclass of TextView that includes rich editing capabilities.
3. **Button:** A push-button that can be pressed, or clicked, by the user to perform an action.
4. **AutoCompleteView:** Similar to EditText, except that it shows a list of completion suggestions automatically while the user is typing.

Input Controls (cont'd)

5. **ImageButton:** This shows a button with an image (instead of text) that can be pressed or clicked by the user.
6. **CheckBox:** An on/off switch that can be toggled by the user. You should use check box when presenting users with a group of selectable options that are not mutually exclusive.
7. **RadioButton:** The RadioButton has two states: either checked or unchecked.
8. **ProgressBar:** The ProgressBar view provides visual feedback about some ongoing tasks, such as when you are performing a task in the background.
9. **Spinner:** A drop-down list that allows users to select one value from a set.
10. **TimePicker:** The TimePicker view enables users to select a time of the day, in either 24-hour mode or AM/PM mode.
11. **DatePicker:** The DatePicker view enables users to select a date of the day.

Event Handling

Events are the way a User interacts with the Android system like button presses or screen touch etc.

The Android framework maintains an event queue as first-in, first-out (FIFO) basis. You can capture these events in your program and take appropriate action as per requirements.

There are three concepts related to Android Event Management.

- **Event Listeners** – An event listener is an interface in the View class that contains a single callback method.
- **Event Listeners Registration** – Event Registration is the process by which an Event Handler gets registered with an Event Listener so that the handler is called when the Event Listener fires the event.

Event Handling (cont'd)

- **Event Handlers** – When an event happens and we have registered an event listener for the event, the event listener calls the **Event Handlers**, which is the method that actually handles the event.

Two ways of Handling events:

1. Using Event Listeners

```
public class ExampleActivity extends Activity implements OnClickListener {  
    protected void onCreate(Bundle savedInstanceState) {  
        ...  
        Button button = (Button)findViewById(R.id.corky);  
        button.setOnClickListener(this);  
    }  
  
    // Implement the OnClickListener callback  
    public void onClick(View v) {  
        // do something when the button is clicked  
    }  
    ...  
}
```

Event Handling (cont'd)

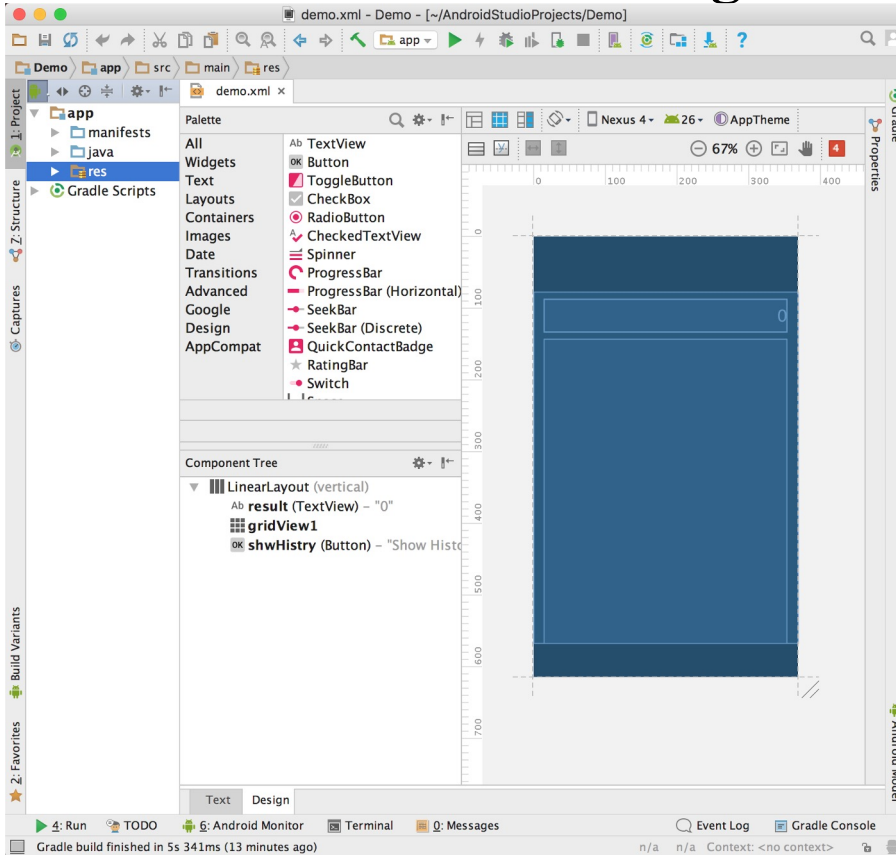
2. Using Event Handler (after declaring method name in the XML)

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
... xmlns:app="http://schemas.android.com/apk/res-auto"
... xmlns:tools="http://schemas.android.com/tools"
... android:layout_width="match_parent"
... android:layout_height="match_parent"
... tools:context="android.demo.marco.papa.com.demo.ServiceActivity">

... <Button
...     android:id="@+id/button2"
...     android:layout_width="match_parent"
...     android:layout_height="wrap_content"
...     android:layout_margin="10dp"
...     android:layout_marginBottom="20dp"
...     android:layout_marginTop="30dp"
...     android:background="@android:color/ho_lo_purple"
...     android:onClick="startService"
...     android:text="Start Services"
...     tools:layout_editor_absoluteX="8dp"
...     tools:layout_editor_absoluteY="0dp" />
```

```
public class ServiceActivity extends AppCompatActivity {
    ... String msg = "Android : ";
    ... /** Called when the activity is first created. */
    ... @Override
    ... public void onCreate(Bundle savedInstanceState) {
    ...     super.onCreate(savedInstanceState);
    ...     setContentView(R.layout.activity_service);
    ...     setTitle("Service");
    ...     Log.d(msg, "The onCreate() event");
    ... }
    ... public void startService(View view) {
    ...     startService(new Intent(getApplicationContext(), MyService.class));
    ... }
}
```

Design UI in XML

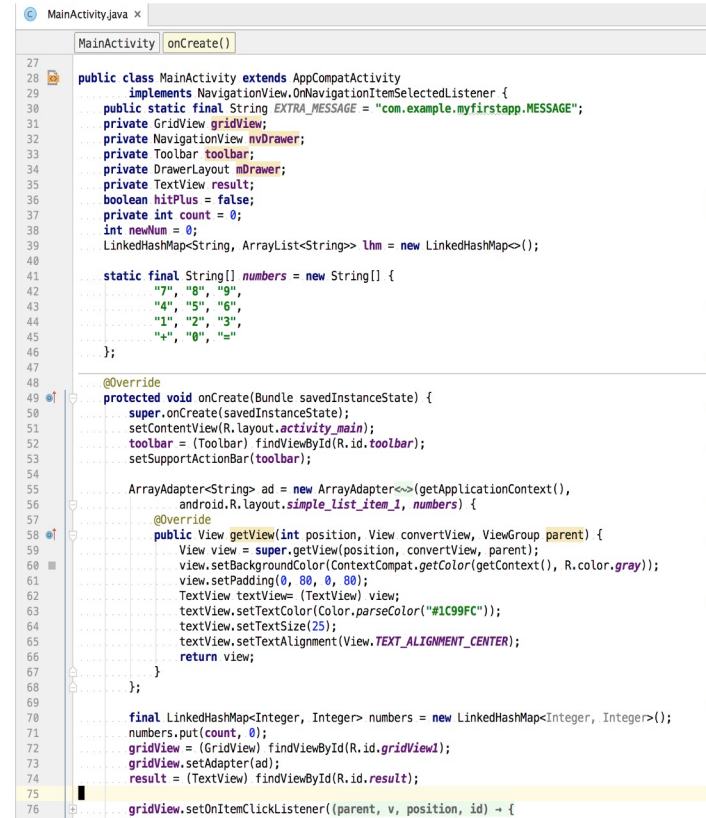


Calculator Demo - Control UI & Logic using Activity

setContentViews() sets the first UI -
activity_main.xml

Steps to produce a Grid UI:

1. Define an ArrayAdapter which would contain a list of items.
2. findViewById(R.id.gridView1) is used to connect UI elements with ID – gridView1 with the Java code.
3. Set the adapter in the Grid view using the code gridView.setAdapter(ad);
4. gridView.setOnItemClickListener() is the click listener for the grid items of the calculator.



```
27 MainActivity.java x
28 MainActivity onCreate()
29
30 public class MainActivity extends AppCompatActivity
31     implements NavigationView.OnNavigationItemSelectedListener {
32     public static final String EXTRA_MESSAGE = "com.example.myfirstapp.MESSAGE";
33     private GridView gridView;
34     private NavigationView nvDrawer;
35     private Toolbar toolbar;
36     private DrawerLayout mDrawer;
37     private TextView result;
38     boolean hitPlus = false;
39     private int count = 0;
40     int newNum = 0;
41     LinkedHashMap<String, ArrayList<String>> lhm = new LinkedHashMap<>();
42
43     static final String[] numbers = new String[] {
44         "7", "8", "9",
45         "4", "5", "6",
46         "1", "2", "3",
47         "4", "0", "="
48     };
49
50     @Override
51     protected void onCreate(Bundle savedInstanceState) {
52         super.onCreate(savedInstanceState);
53         setContentView(R.layout.activity_main);
54         toolbar = (Toolbar) findViewById(R.id.toolbar);
55         setSupportActionBar(toolbar);
56
57         ArrayAdapter<String> ad = new ArrayAdapter<>(getApplicationContext(),
58             android.R.layout.simple_list_item_1, numbers) {
59             @Override
60             public View getView(int position, View convertView, ViewGroup parent) {
61                 View view = super.getView(position, convertView, parent);
62                 view.setBackgroundColor(ContextCompat.getColor(getContext(), R.color.gray));
63                 view.setPadding(0, 80, 0, 80);
64                 TextView textView = (TextView) view;
65                 textView.setTextColor(Color.parseColor("#1C99FC"));
66                 textView.setTextSize(25);
67                 textView.setTextAlignment(View.TEXT_ALIGNMENT_CENTER);
68                 return view;
69             }
70         };
71
72         final LinkedHashMap<Integer, Integer> numbers = new LinkedHashMap<Integer, Integer>();
73         numbers.put(count, 0);
74         gridView = (GridView) findViewById(R.id.gridView1);
75         gridView.setAdapter(ad);
76         result = (TextView) findViewById(R.id.result);
77
78         gridView.setOnItemClickListener((parent, v, position, id) -> {
```

Control UI & Logic using Activity

Logic for Calculator:

- While consecutive numbers are pressed, treat them as part of a single number by multiplying the term by 10 every time.
- While “+” is clicked, keep updating the UI with the current number.
- When “=” is clicked, it marks the end of an addition. Add up all the numbers and display in the result bar.
- Add all the queries to a HashMap to display it in the History section.
- History is temporary.

```
MainActivity.java x
MainActivity onCreate()
76 ..... gridView.setOnItemClickListener((parent, v, position, id) -> {
77 .....     switch(((TextView) v).getText().toString()){
78 .....         case "=":
79 .....             int sum = 0;
80 .....             Set set = numbers.entrySet();
81 .....             Iterator i = set.iterator();
82 .....             ArrayList<String> lhmList = new ArrayList<String>();
83 .....             while(i.hasNext()) {
84 .....                 Map.Entry me = (Map.Entry) i.next();
85 .....                 int value = (int)me.getValue();
86 .....                 sum += value;
87 .....                 lhmList.add(Integer.toString(value));
88 .....             }
89 .....             lhmList.add(Integer.toString(sum));
90 .....             lhm.put(Integer.toString(count), lhmList);
91 .....             numbers.clear();
92 .....             result.setText(Integer.toString(sum));
93 .....             hitPlus = false;
94 .....             break;
95 .....         case "0":
96 .....         case "1":
97 .....         case "2":
98 .....         case "3":
99 .....         case "4":
100 .....         case "5":
101 .....         case "6":
102 .....         case "7":
103 .....         case "8":
104 .....         case "9":
105 .....             if(hitPlus == false && numbers.size() != 0){
106 .....                 int oldValue = numbers.get(count);
107 .....                 newNum = (oldValue * 10) + Integer.parseInt(((TextView) v).getText().toString());
108 .....                 numbers.put(count, newNum);
109 .....             } else {
110 .....                 newNum = Integer.parseInt(((TextView) v).getText().toString());
111 .....                 numbers.put(++count, newNum);
112 .....             }
113 .....             result.setText(Integer.toString(newNum));
114 .....             hitPlus = false;
115 .....             break;
116 .....         case "+":
117 .....             hitPlus = true;
118 .....             result.setText("");
119 .....             Toast.makeText(getApplicationContext(),
120 .....                 "+", Toast.LENGTH_SHORT).show();
121 .....             break;
122 .....     }
123 ..... }
124 .....
125 .....
126 ..... });
127 .....
128 .....
129 ..... mDrawer = (DrawerLayout) findViewById(R.id.drawer_layout);
130 ..... ActionBarDrawerToggle toggle = new ActionBarDrawerToggle(
131 .....     this, mDrawer, toolbar, "Open navigation drawer", "Close navigation drawer");
132 ..... mDrawer.setDrawerListener(toggle);
133 ..... toggle.syncState();
134 ..... }
```

Change Activity properties in Manifest file

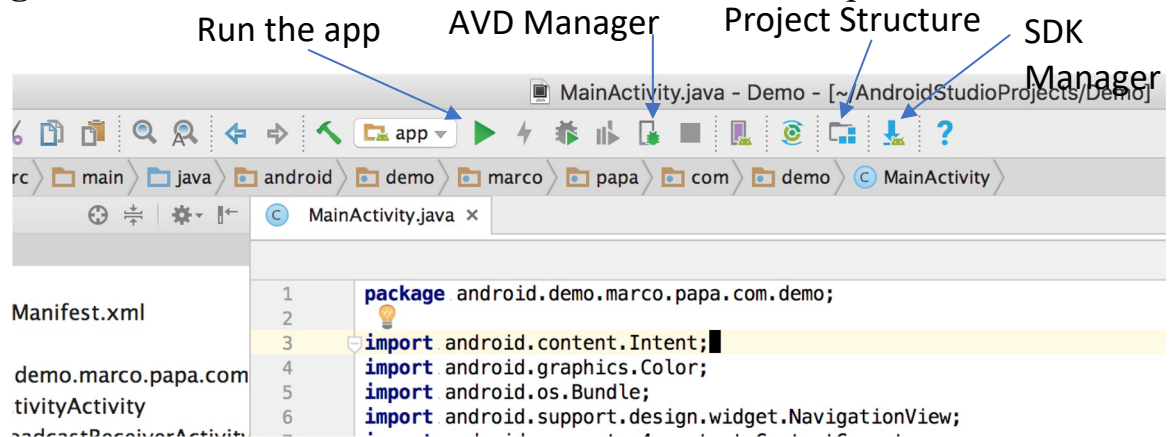
1. Every Activity that is created must be defined in the Manifest file under application tag.
2. The **action** for the intent filter indicates that this activity serves as the entry point for the application.
3. The **category** for the intent-filter indicates that the application can be launched from the device's launcher icon.
4. Parent activity can be shown for the up-arrow navigation.

Run your app in the Emulator

To run your app in your own Android device, tap on Build Number from Settings several times. This will enable developer options in your phone. Then enable USB debugging.

To run your app in the Emulator, make sure you have correct SDK & tools installed. Verify using the following steps:

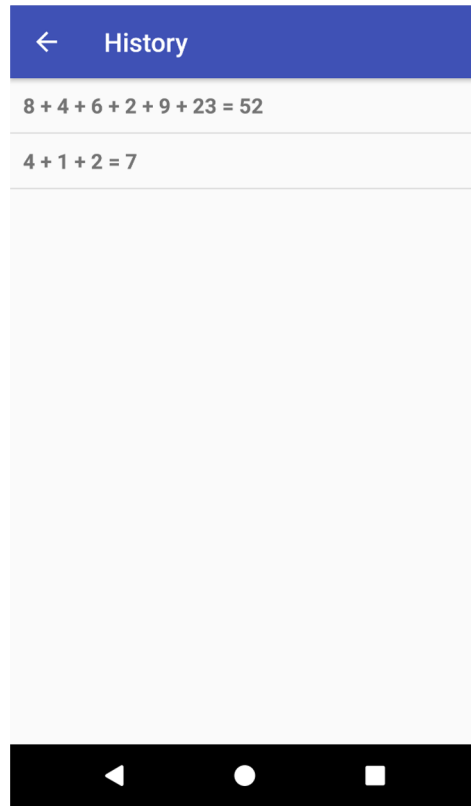
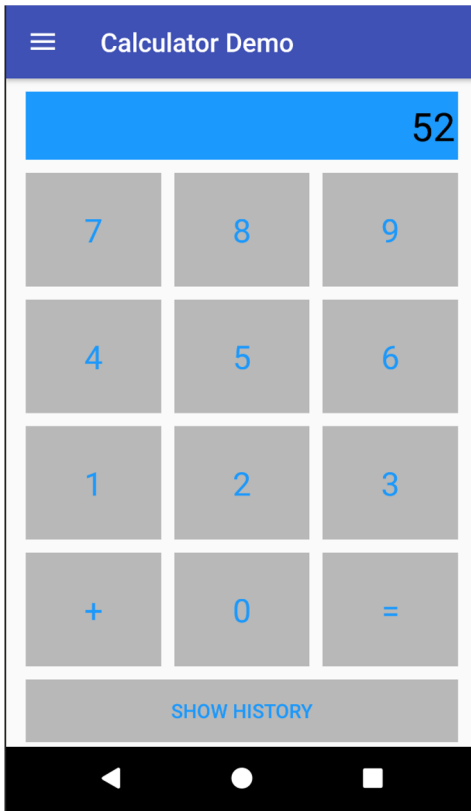
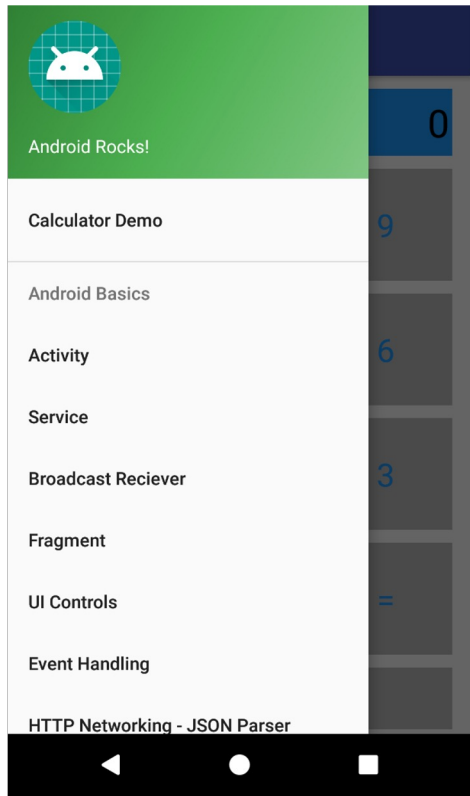
1. Click on the Project Structure icon and then click on the “app” section under Modules, or right-click on **app**, and select **Open Module Settings**.
 1. Under **Properties**, check the **Compile Sdk Version** and **Build Tools Version**.
 2. Under **Default Config**, **Min SDK Version** should be lower than the Compile SDK Version.



Run your app in the Emulator (cont'd)

2. Click on **Tools** -> **SDK Manager** and then **Languages & Frameworks** -> **Android SDK** from the left menu.
 1. Under **SDK Platforms**, make sure the Compile Android version you selected in the last step is installed (under **Status**).
 2. Under **SDK tools**, make sure **Android Emulator**, **Android SDK Platform-Tools**, **Android SDK Build-Tools** & Intel x86 Emulator Accelerator are all installed.
3. Click on **Tools** -> **Device Manager** and then click on '+' (Create Virtual Device) to fire up a new Emulator.
 1. Under **Phone** category, select the profile (mobile type) that you wish to start. Click **Next**.
 2. Under **Recommended** section, make sure the Release is already downloaded. If not, download it. Click **Next**.
 3. Check the **Device Name**. That will be the name of the device shown every time you start Emulator. Click **Finish**.
 4. Now you will see the same device under **Name**. Click on the **green play** button on the right to start the Emulator.

Calculator Demo



Working with 3rd party libraries

- There are many times when we need to depend on 3rd party libraries to achieve functionality we want from our app.
- Using 3rd party libraries helps save time as we don't have to reinvent the wheel when somebody else has done it for you. This also saves a lot on debugging time.
- We often use **Gradle** to add the libraries as dependencies to our project (although it is possible to manually download and include the library, but this is not preferred).
- Gradle is a flexible tool that allows us to configure how our project is compiled, built and packaged into our final app.
- Apart from managing dependencies, Gradle is also used to manage different build variants (such as a debug version with logging enabled vs a release version) or project flavors (such as free vs paid versions of the app).
- Latest version is **Android Gradle plugin 8.7**
- Release Docs: <https://developer.android.com/studio/releases/gradle-plugin>

Using Gradle for dependency management

- It is fairly simple to add third party libraries to our project via Gradle.
- We need to add the following in the build.gradle file for our app module.

```
dependencies {  
    ...  
    implementation 'library_package_name:library:version_code'  
}
```

- We can find the above details on the webpage of the library we want to use.
- The build system will automatically find these libraries, download them and include them in your project while building your project. You need not manually download the libraries into your project.
- Note: In many examples online you may find ‘compile’ instead of ‘implementation’. Compile was recently deprecated in favor of implementation, but both should work in most cases.

Common 3rd party libraries

- In most projects, the following libraries are useful with simplifying common tasks:
 - Volley
 - Picasso
 - Glide
 - Gson
 - MPAndroidChart
 - Android HighCharts

Volley - HTTP Networking and JSON Parsing

- Volley is an HTTP library that makes networking for Android apps easier and most importantly, faster. Volley is available on [GitHub](https://github.com/google/volley).

<https://github.com/google/volley>

Two ways to include Volley in your project:

1. The easiest way to add Volley to your project is to add the following dependency to your app's build.gradle file:

```
dependencies {  
    ...  
    compile 'com.android.volley:volley:1.0.0'  
}
```

1. You can also clone the Volley repository and set it as a library project:
 1. Git clone the repository by typing the following at the command line:
git clone <https://github.com/google/volley>
 2. Import the downloaded source into your app project as an Android library module.

Volley - HTTP Networking and JSON Parsing (cont'd)

To use Volley, you must add the `android.permission.INTERNET` permission to your app's manifest. Without this, your app won't be able to connect to the network.

```
manifest
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="android.demo.marco.papa.com.demo">
    ...
    <uses-permission android:name="android.permission.INTERNET">
    </uses-permission>
    ...
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
```

Volley at work

```
... @Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_http);
    setTitle("HTTP Networking & JSON Parser");

    textView = (TextView) findViewById(R.id.volleyText);
    responseName = (TextView) findViewById(R.id.responseName);
    requestQueue = Volley.newRequestQueue(this); // 'this' is the Context
    String url = "https://jsonplaceholder.typicode.com/users";

    JSONArrayRequest jsonArrayRequest = new JSONArrayRequest(Request.Method.GET, url, null,
        new Response.Listener<JSONArray>() {
            @Override
            public void onResponse(JSONArray response) {
                textView.setText("Trimmed response: " + response.toString());
                Toast.makeText(getApplicationContext(), response.toString(), Toast.LENGTH_SHORT);
                StringBuilder names = new StringBuilder();
                names.append("Parsed names from the response: ");
                try {
                    for(int i = 0; i < response.length(); i++){
                        JSONObject jsonResponse = response.getJSONObject(i);
                        String name = jsonResponse.getString("name");
                        names.append(name).append(", ");
                        Log.d("Name", name);
                    }
                    names.deleteCharAt(names.length() -2);
                    responseName.setText(names.toString());
                } catch (JSONException e) {
                    e.printStackTrace();
                }
            }
        },
        new Response.ErrorListener() {
            @Override
            public void onErrorResponse(VolleyError error) {
                Toast.makeText(getApplicationContext(), "Nothing found!", Toast.LENGTH_SHORT);
            }
        });
    //add request to queue
    requestQueue.add(jsonArrayRequest);
}
}
```

Picasso – Image downloading and caching

- Today's apps are rich in images that enhance the user interface. But images in apps add many complications.
- Images are large in size and take time to download and then render. If we wait for all images to load before showing anything, it can lead to a bad user experience.
- Sometimes the user may exit the screen before the image finishes downloading. We need to intelligently cancel the image download in this case.
- The same images are often shown multiple times and need to be cached to improve performance.
- Often we need to transform our images after downloading (like cropping and resizing).
- We also need to handle cases where the image download fails and placeholder images are needed while the actual image loads.
- More info at: <https://square.github.io/picasso/>

Picasso – Image downloading and caching

- We can easily add Picasso to our project using Gradle.

```
implementation 'com.squareup.picasso:picasso:2.71828'
```

- We can then import the Picasso class into the .java file where we wish to Picasso to load images.

```
import com.squareup.picasso.Picasso;
```

- For most cases, this one line of code is all we need to load our image:

```
Picasso.with(context).load("http://ourdomainname/image.png")  
    .into(imageView);
```

This takes care of asynchronously downloading the image, caching and auto-canceling download when no longer needed.

Picasso – Image downloading and caching

- Simple methods also exist to resize, rotate and crop the image.

```
Picasso.with(context).load("http://ourdomainname/image.png")
    .resize(width, height)
    .rotate(degrees)
    .centerCrop()
    .into(imageView);
```

- Picasso also provides methods for placeholder images while our image loads and for the case where our image download failed.

```
Picasso.with(context).load("http://ourdomainname/image.png")
    .placeholder(R.drawable.placeholder_image)
    .error(R.drawable.error_image)
    .into(imageView);
```

Glide - Image downloading and caching

- Another library like Picasso for downloading and caching images and GIFs.
- Installation:

```
dependencies {  
    implementation 'com.github.bumptech.glide:glide:4.9.0'  
    annotationProcessor 'com.github.bumptech.glide:compiler:4.9.0'  
}
```

Usage:

```
Glide.with(myFragment).load(url).centerCrop()  
    .placeholder(R.drawable.loading_spinner).into(myImageView);
```

- Compatible with Volley HTTP calls
- More info at: <https://github.com/bumptech/glide>

Gson – serializing and desterializing Java objects

- **Gson** is a library that is used to convert Java objects into their JSON representation and vice-versa.
- With this you can easily convert your Java objects to their JSON string representations. This is very useful when you need to send data between activities or when you need to store an object for it to persist the next time you use the app.
- Another use case is for communicating with APIs that send/receive JSON data.
- This saves us from writing code to parse our JSON into Java objects and vice versa. We only need to define the model of the data that we expect to receive and the library will take care of the parsing.
- More info at: <https://github.com/google/gson>
<https://www.baeldung.com/kotlin/gson-parse-arrays>

Gson – serializing and deserializing Java objects

- We can easily add Gson to our project using Gradle.

```
implementation 'com.google.code.gson:gson:2.8.2'
```

- Lets assume we get this JSON object from our API:

```
[
  {
    "name": "John Doe",
    "email_id": john@doe.com
  },
  {
    "name": "Jane Doe",
    "email_id": jane@doe.com
  }
]
```

Gson – serializing and deserializing Java objects

Let's use the following class to store data received from our API:

```
//this class defines the model of our JSON data
public class Contact {
    //we can use the same name as the field in the JSON object
    //no special annotation is needed in this case
    String name;

    //We can also use our names using the
    //@SerializedName annotation
    @SerializedName("email_id") //name of field in JSON object
    String emailID; //our object name
}
```

Gson – serializing and deserializing Java objects

- In our onResponse() method for our Volley HTTP request, we can directly parse the response into an array of Java objects

```
JsonObjectRequest jsonObjectRequest = new
JsonObjectRequest(Request.Method.GET, url, null,
    new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            Contact[] contacts = (Contact[])
                gson.fromJson(response, Contact[].class);
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            //...
        }
    });
```

- Like so, we have directly parsed our JSON response.

MPAndroidChart – Open Source graphing Library

- "A powerful and easy to use chart library for Android". Open Source, can be found at: <https://github.com/PhilJay/MPAndroidChart>
- Used to provide various charts like LineChart, BarChart, PieChart
- Create a List of Entry objects where each Entry is X vs Y values. Convert each EntryList to a DataSet and call the charting method.

MPAndroidChart
created by Philipp Jahoda

- Dependency and usage:

```
dependencies { implementation 'com.github.PhilJay:MPAndroidChart:v3.1.0' }  
List<Entry> entries1 = new ArrayList<>();  
List<Entry> entries2 = new ArrayList<>();  
for (int i = 0; i < daily.length(); i++) {  
    Float fmin = ...; // Minimum temperature from json  
    Float fmax = ; // Maximum temperature from json  
    entries1.add(new Entry(i, fmin));  
    entries2.add(new Entry(i, fmax));  
}
```

MPAndroidChart – Usage

```
//Create a LineDataSet for each LineChart and change properties
LineDataSet dataSet1 = new LineDataSet(entries1, "Minimum Temperature");
dataSet1.setColor(Color.parseColor("#BB86FC"));
LineDataSet dataSet2 = new LineDataSet(entries2, "Maximum Temperature");
dataSet2.setColor(Color.parseColor("#FAAB1A"));
//Render the chart
List<ILineDataSet> dataSets = new ArrayList<>();
dataSets.add(dataSet1);
dataSets.add(dataSet2);
LineData data = new LineData(dataSets);
LineChart chart = (LineChart) view.findViewById(R.id.chart);
chart.setData(data);
chart.invalidate(); // refresh
```

To change the style use:

`Chart.getLegend().set__()` and `chart.getAxis__().set__()` methods

HighCharts Android – Graphing Library

- A powerful and easy to use chart library for “Android”. More info at: <https://www.highcharts.com/blog/integrations/android/>
- Offers the capability to initialize a chart as a normal “view” in Android.

- Dependencies and Usage



```
dependencies {  
    implementation 'com.github.highcharts:highcharts-android:9.3.2'  
}
```

- Usage

```
<!-- Define a HighChart view - - >
```

```
<com.highsoft.highcharts.core.HIChartView  
    android:id="@+id/hc"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent" />
```

Android HighCharts – Usage

```
//Create a HighChartView object.
```

```
HIChartView chartView = (HIChartView) findViewById(R.id.hc);
```

```
//Specify chart options with HIOptions
```

```
HIOptions options = new HIOptions();
```

```
// Create a HIChart, specify the type of chart and add to options
```

```
HIChart chart = new HIChart();
```

```
chart.setType("column");
```

```
options.setChart(chart);
```


Android HighCharts – Usage (cont'd)

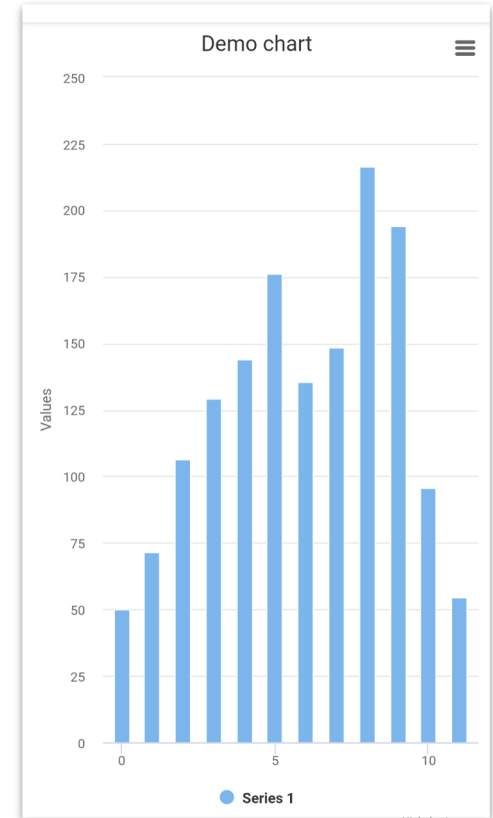
```
//Set title of that chart
HITitle title = new HITitle();
title.setText("Demo chart");
options.setTitle(title);

//Now create a column and add data
HIColumn series = new HIColumn();
series.setData(new ArrayList<>(Arrays.asList(49.9, 71.5, 106.4, 129.2,
144, 176, 135.6, 148.5, 216.4, 194.1, 95.6, 54.4))));

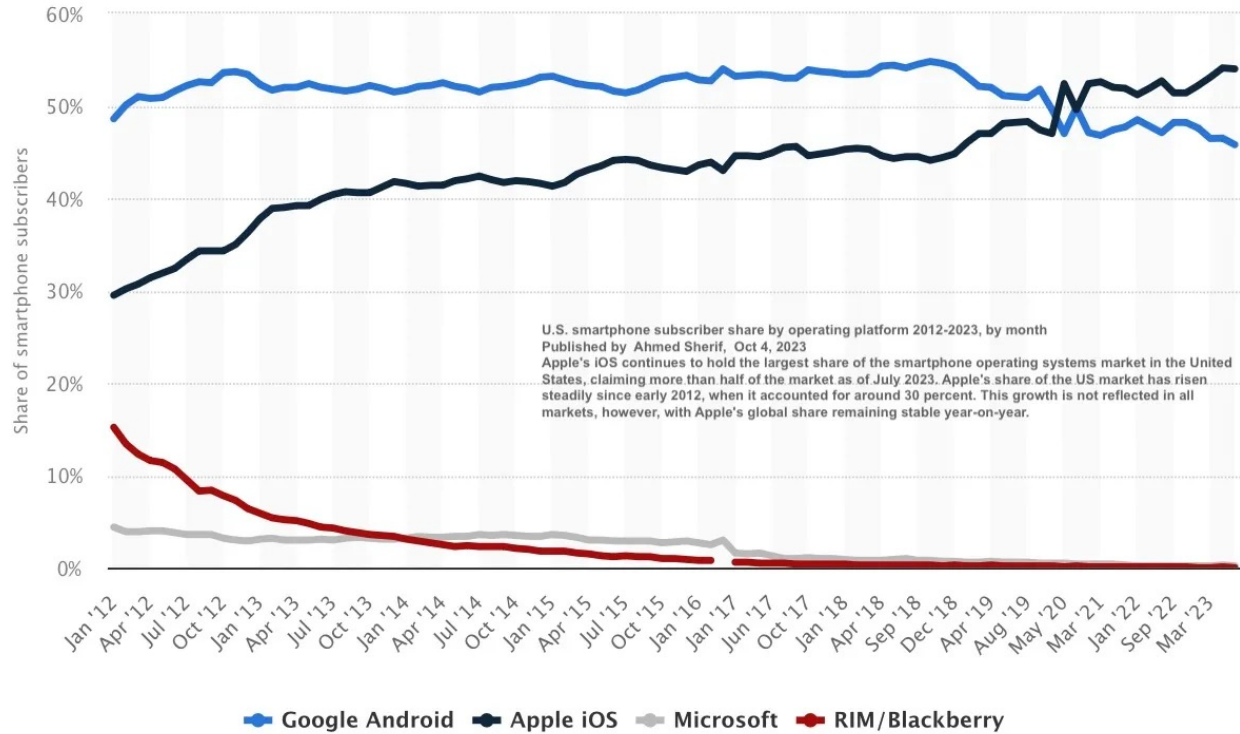
//Add the column to the chart options
options.setSeries(new
    ArrayList<HISeries>(Collections.singletonList(series)));
```

Android HighCharts – Usage (cont'd)

```
//At last, set options to the chart view  
chartView.setOptions(options);
```



Android vs iOS Subscriber share



Modern Android Development Overview

- Languages: Kotlin vs Java
- UI & Layout: Jetpack Compose vs XML

Kotlin vs Java

- Like Swift vs Objective-C
- New default (de-facto) Android language
- Provides a lot of newer concepts: null safety, coroutines, lambdas, data classes...
- Fully interoperable with Java (and vice-versa) => can gradually migrate codebase
- Less wording, more syntax sugar
- See <https://kotlinlang.org/>

```
fun main() {  
    val numbers = listOf(1, -2, 3, -4, 5, -6)  
    val positives = numbers.filter { it > 0 }  
    val hasNegatives = numbers.any { c -> c < 0 }  
    println("Positive numbers: $positives")  
    println("Is there any number less than 0: $hasNegatives")  
}
```

```
data class User(val name: String, val id: Int) {  
    override fun equals(other: Any?) =  
        other is User && other.id == this.id  
}  
  
fun main() {  
    val user = User( name: "Alex", id: 1)  
    println(user)  
  
    val secondUser = User( name: "Alex", id: 1)  
    println("user == secondUser: ${user == secondUser}")  
}
```

Jetpack Compose

- See:
<https://developer.android.com/compose>
- Similar to Flutter, React Native, Swift UI, etc.
- Alternative approach to build UI – tree of widgets defined by code
- Works with Kotlin
- Provides tons of simple ways to build complex features like animations, etc.

See compose online tutorial:

<https://developer.android.com/courses/jetpack-compose/course>

```
@Composable
fun DialExample(
    onConfirm: () -> Unit,
    onDismiss: () -> Unit,
) {
    val currentTime = Calendar.getInstance()

    val timePickerState = rememberTimePickerState(
        initialHour = currentTime.get(Calendar.HOUR_OF_DAY),
        initialMinute = currentTime.get(Calendar.MINUTE),
        is24Hour = true,
    )

    Column {
        TimePicker(
            state = timePickerState,
        )
        Button(onClick = onDismiss) {
            Text("Dismiss picker")
        }
        Button(onClick = onConfirm) {
            Text("Confirm selection")
        }
    }
}
```

References

- <https://developer.android.com/studio/intro/index.html>
- <https://www.tutorialspoint.com/android>
- <https://www.javatpoint.com/android-tutorial>
- <http://socialcompare.com/en/comparison/android-versions-comparison>
- <http://square.github.io/picasso/>
- <https://github.com/google/gson>
- <https://kylewbanks.com/blog/tutorial-parsing-json-on-android-using-gson-and-volley>
- <https://github.com/highcharts/highcharts-android>